

When Do Larger Batches Help Scale LLM Reinforcement Learning?

Ziniu Li Jinbo Wang Guanhua Huang
Feiyuan Zhang Pengbo Li Alex Chen

Tencent Hunyuan Team

ziniuli@tencent.com

Abstract

Larger batches reduce the variance of stochastic gradients per update and are therefore often expected to accelerate training. Yet whether this statistical benefit translates into lower wall-clock time-to-target remains unclear, because each update consumes more samples and may take longer to execute. We study this tradeoff in reinforcement learning for large language models. We separate its *algorithmic* and *systems* effects by comparing learning and execution along their natural axes. At the algorithmic level, we compare configurations at equal *cumulative* sample counts while retuning batch-dependent hyperparameters. Over a bounded range of batch sizes, this procedure yields an approximately *batch-size-invariant* family whose members follow similar sample-indexed learning trajectories. At the systems level, we exploit the computational *asymmetry* between rollout generation and training: autoregressive generation is often memory-bandwidth-bound at low concurrency, whereas training work scales approximately with the number of processed tokens. Combining these two views yields a direct decision rule: a larger-batch configuration reduces time-to-target only when its throughput gain exceeds its samples-to-target penalty. Experiments with GRPO and PPO support both sides of this decomposition. At the algorithmic level, square-root learning-rate scaling with Adam produces approximately batch-size-invariant learning curves over a bounded range of batch sizes. At the systems level, larger batches improve generation throughput by up to $2.29\times$ on fixed hardware. In GRPO, combining higher throughput with learning-rate retuning reduces time-to-target by up to 29%, whereas increasing the batch without retuning is slower despite its higher throughput.

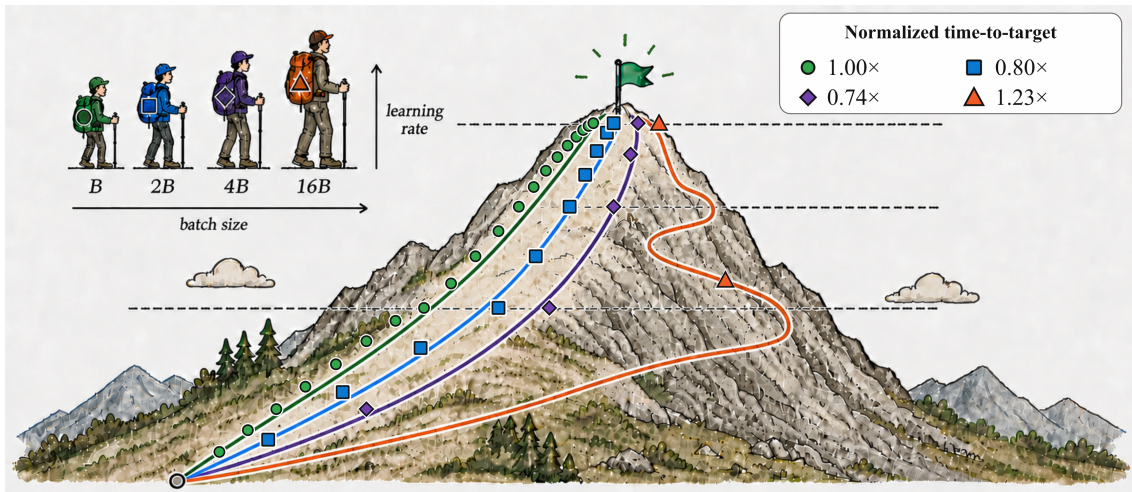


Figure 1: A data-informed schematic based on selected GRPO operating points from our experiments (Table 1). Each trail marker denotes one optimizer update. It highlights three messages: (1) after learning-rate retuning, B , $2B$, and $4B$ are approximately batch-size invariant in cumulative samples; (2) higher realized throughput within this regime improves wall-clock efficiency; and (3) beyond this regime, as at $16B$, normalized time-to-target deteriorates, rising above the B baseline.

1 Introduction

Scaling reinforcement learning (RL) to larger language models (LLMs) and longer training runs demands substantially more computation. Recent reasoning systems such as OpenAI o1 and DeepSeek-R1 illustrate the growing role of RL post-training in improving LLM reasoning (OpenAI, 2024; Guo et al., 2025). Increasing the batch size is a primary means of exposing parallelism: larger batches place more rollout-generation and optimization work in flight, potentially improving hardware utilization while reducing stochastic-gradient variance (McCandlish et al., 2018; Shallue et al., 2019). However, each update also consumes more newly generated samples and may take longer to collect and optimize. This raises a fundamental question: can large-batch training actually reduce wall-clock time-to-target?

Prior studies (Goyal et al., 2017; Smith et al., 2018; Shallue et al., 2019) suggest that data-parallel scaling is most effective when two conditions hold: (1) additional computational resources allow larger batches to be processed through data parallelism while keeping per-step time approximately constant; and (2) the batch remains within the statistically efficient regime, typically below or near a threshold commonly referred to as the *critical training batch size*, with batch-dependent hyperparameters appropriately retuned. In this paper, we instead study this question in the practically important *fixed-hardware* setting¹: **can increasing the batch size alone reduce wall-clock time-to-target, and if so, under what conditions?** In conventional supervised learning, this opportunity is often limited once dense forward-backward computation saturates the available hardware. LLM RL has a different computational structure: training samples must first be generated, typically through autoregressive decoding, before they can be consumed by model updates (Li et al., 2024; Shao et al., 2024). Consequently, batch tuning couples an *algorithmic* question (how much learning is obtained from a fixed number of samples) with a *systems* question (how quickly those samples are generated and consumed).

To formalize the tradeoff, we use a two-level learning-effectiveness-throughput decomposition:

$$\underbrace{\frac{dJ}{dt}}_{\text{learning progress rate}} = \underbrace{\frac{dJ}{dN}}_{\text{sample efficiency}} \times \underbrace{\frac{dN}{dt}}_{\text{systems efficiency}}, \quad (1)$$

where J denotes evaluation performance, N denotes cumulative consumed samples, and t denotes wall-clock time. The first factor measures learning progress per sample and captures the algorithmic effect of batch size; the second measures the rate at which samples are generated and consumed and captures its systems effect. To compare batch configurations, we fix a performance target. For each configuration, let the *samples-to-target* N^* be the cumulative number of samples required to reach it, and let T be the corresponding wall-clock time-to-target. The realized average end-to-end throughput over that interval is then $q = N^*/T$. Thus, comparing a larger-batch configuration 2 with a reference configuration 1,

$$T_2 < T_1 \iff \frac{q_2}{q_1} > \underbrace{\frac{N_2^*}{N_1^*}}_{r_N}, \quad (2)$$

where r_N is the samples-to-target penalty.

Equation (2) answers the title question: a larger batch helps precisely when its throughput gain exceeds its sample penalty. In practice, we can implement this decision rule in two stages. First, we retune batch-dependent hyperparameters to construct an approximately *batch-size-invariant* family, for which $r_N \approx 1$ over a bounded regime (McCandlish et al., 2018; Hilton et al., 2022; Malladi et al., 2022). Second, we exploit generation-training asymmetry to increase q . In bandwidth-bound decoding, model weights are streamed once per step, so larger active batches amortize this nearly fixed traffic across more tokens. Generation time can therefore grow *sublinearly* with batch size while training work grows approximately linearly with processed tokens (Williams et al., 2009; Pope et al., 2023; Zhong et al., 2024). This asymmetry can increase end-to-end throughput.

Empirically, we study GRPO with Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025) and PPO with an internal Hunyuan mixture-of-experts model with 3B active parameters. Figure 1 provides a schematic preview. First, square-root learning-rate scaling for Adam produces approximately batch-size-invariant learning curves in both workloads. In particular, this invariance holds only over a *bounded* range and begins to break down at extremely larger batch sizes. Further, this alignment also depends on batch-dependent learning-rate retuning: in our GRPO fixed-learning-rate control, doubling the batch increases samples-to-target by 67%. Second, we show that increasing the PPO batch by $4\times$ on fixed

¹Focusing on fixed hardware isolates the effect of batch size from that of adding accelerators. The same throughput-versus-sample-efficiency criterion also applies when the accelerator count changes, after accounting for the corresponding change in throughput.

hardware improves generation throughput by $2.29\times$. In GRPO, larger batches with learning-rate retuning reduce time-to-target by up to 29%. Together, these results demonstrate that fixed-hardware speedup is conditional on both preserving learning per sample and improving systems throughput.

To summarize, our contributions are:

- We formulate batch-size invariance as a comparison and transfer principle and study it empirically across PPO and GRPO in modern LLM RL post-training.
- We characterize the generation–training asymmetry that enables acceleration even under a fixed hardware allocation and measure how rollout-collection cost changes with batch size.
- We operationalize the decision rule as a practical two-stage tuning procedure. Our experiments support both components and both directions of the rule: generation throughput improves by up to $2.29\times$, tuned larger-batch configurations reduce time-to-target by up to 29%, and a fixed-learning-rate control is slower despite higher throughput.

2 Problem Formulation

The criterion in Equation (2) combines a samples-to-target ratio with a throughput ratio. We therefore begin by fixing the sample-accounting rule and distinguishing the batch quantities that affect learning from those that affect execution. At a rollout-collection boundary, let θ denote the current policy parameters. The policy π_θ samples G responses for each of P prompts,

$$y_{i,j} \sim \pi_\theta(\cdot \mid x_i), \quad i \in \{1, \dots, P\}, \quad j \in \{1, \dots, G\}.$$

After a reward model or verifier scores these rollouts, the algorithm constructs returns or advantage estimates. PPO trains an actor together with a learned value model (Schulman et al., 2017). Value-model-free, REINFORCE-style methods such as ReMax instead avoid training a separate critic and construct a variance-reduction baseline from sampled rewards (Williams, 1992; Li et al., 2024), while GRPO uses responses to the same prompt to form a group-relative baseline (Shao et al., 2024).

This notation covers one-response objectives such as PPO and ReMax with $G = 1$, as well as grouped objectives such as GRPO with $G > 1$. One optimizer update uses a nominal training batch $B_{\text{train}} = PG$. The generation-side batch B_{gen} equals PG in a coupled pipeline but may be larger under asynchronous streaming. Micro-batching and gradient accumulation affect execution only.

We use N for cumulative training samples. For an *algorithmic configuration*—the training batch and its batch-dependent hyperparameters, including the learning rate—the sample-indexed curve $J(N)$ defines samples-to-target N^* . A *systems realization*—generation concurrency, placement, runtime, and hardware—determines the realized end-to-end throughput $q = N^*/T$, and hence $T = N^*/q$. We use q_{gen} specifically for generation-stage throughput.

3 Training-Side Comparability: Invariance and the Critical Batch

Recall the framework we follow in this paper: Equation (1) decomposes wall-clock learning progress into learning per sample and systems throughput. Determining whether a larger batch improves wall-clock progress therefore requires separating its training-side sample efficiency from its systems-side throughput. In particular, comparisons at equal optimizer updates are inappropriate because different batch sizes correspond to different sample budgets: after k updates, a run with batch size $2B$ has processed $2kB$ samples, whereas a run with batch size B has processed only kB . Better performance in the larger-batch run may therefore reflect greater sample exposure rather than higher sample efficiency.

These considerations motivate *batch-size invariance*: after admissible retuning, learning trajectories should remain approximately aligned when indexed by cumulative samples, even though larger batches require fewer sequential updates. In this regime, samples-to-target are approximately preserved, so $r_N \approx 1$, and any wall-clock gain can be attributed to improved throughput. We next formalize this condition and describe how we construct and test it.

3.1 Batch-Size Invariance and Transfer

Throughout this section, B denotes the training batch. After applying a batch-dependent retuning rule, let $J_B(N)$ denote the expected evaluation performance after N cumulative training samples. Batch-size invariance asks whether the hyperparameters can be retuned as B changes so that these sample-indexed trajectories remain approximately aligned (Hilton et al., 2022).

Definition 1 (Approximate batch-size invariance). *Fix a model, task, algorithm, reward definition, evaluation protocol, reference batch B_0 , an admissible batch-retuning rule, sample interval $\mathcal{N}$, and tolerance $\varepsilon > 0$. The resulting batch-dependent family is approximately invariant if, for every batch B in the family,*

$$\sup_{N \in \mathcal{N}} |J_B(N) - J_{B_0}(N)| \leq \varepsilon.$$

The expectation over rollout, optimization, and evaluation randomness is implicit in J_B .

Performance-level invariance allows distinct optimization paths and learned policies as long as they attain similar expected evaluation performance. Related work likewise distinguishes agreement in loss from agreement in function space when studying batch-size effects in online learning (Vyas et al., 2024). Note that the concept of batch-size invariance is deliberately local: changing the model, data distribution, objective, reward, or training stage defines a new learning problem and requires re-establishing alignment.

Why is this property important in practice? It enables data-parallel speedup without sacrificing learning per sample: under a fixed sample budget, increasing the batch size by a factor of s reduces the number of sequential optimizer updates by the same factor s . For example, Goyal et al. (2017) reduced ResNet-50/ImageNet training from 29 hours with a batch of 256 on 8 GPUs to 1 hour with a batch of 8192 on 256 GPUs, while matching accuracy; at the fixed 90-epoch sample budget, the $32\times$ larger batch required $32\times$ fewer sequential optimizer steps. Taking this idea further, the critical-batch literature asks how far this reduction in sequential optimizer steps can be sustained as batch size increases before diminishing returns set in (McCandlish et al., 2018; Shallue et al., 2019).

3.2 From Invariance to the Critical Training Batch

Batch-size invariance describes the ideal pre-knee regime: if samples-to-target are preserved, a larger batch converts the same sample budget into fewer sequential updates. This gain *cannot* continue indefinitely. Under independent sampling, gradient averaging ideally reduces stochastic-gradient variance as $1/B$. This benefit diminishes once the reducible noise is small relative to the gradient signal, or when correlations and nonstationarity leave residual variation. Further averaging then yields little improvement in update quality. The critical training batch marks this onset of diminishing returns for a declared learning target (McCandlish et al., 2018; Shallue et al., 2019).

Definition 2 (Critical training batch). *Fix a model, task, algorithm, evaluation target J^* , an admissible hyperparameter-retuning protocol, and a nonempty set of admissible training batches $\mathcal{B}$. Let $K^*(B)$ be the minimum expected number of sequential actor updates required for the policy to reach J^* with batch B under that protocol, so $1/K^*(B)$ measures update efficiency. Define*

$$B_{\text{train}}^{\text{crit}} = \min \left\{ B \in \mathcal{B} : \frac{1}{K^*(B)} \geq \frac{1}{2} \sup_{B' \in \mathcal{B}} \frac{1}{K^*(B')} \right\}.$$

For a fixed batch, the sample and update counts satisfy $N^*(B) \approx BK^*(B)$. Within an invariant family, $N^*(B)$ is approximately preserved, and hence

$$K^*(B) \approx \frac{N^*}{B}. \quad (3)$$

Equation (3) predicts that doubling the batch should roughly halve the sequential updates needed to reach the target.

Under the empirical model $K^*(B) = K_{\min}^*(1 + B_{\text{train}}^{\text{crit}}/B)$ of McCandlish et al. (2018), $K_{\min}^*$ is the asymptotic minimum updates-to-target. At $B = B_{\text{train}}^{\text{crit}}$, $K^*(B) = 2K_{\min}^*$, so the definition recovers the conventional half-efficiency turning point when $\mathcal{B}$ spans the large-batch limit.

To instantiate the retuning rule in our experiments, we vary only the learning rate with batch size. For Adam, we initialize the batch-dependent learning rate with the square-root rule motivated by the SDE analysis of Malladi et al. (2022):

$$\eta(B) = \eta_0 \sqrt{\frac{B}{B_0}}.$$

The square-root rule is only an initialization, not a guarantee of invariance. Although prior work suggests that other optimizer- and algorithm-specific hyperparameters may also benefit from batch-dependent adjustment (Goyal et al., 2017; Smith et al., 2018; Malladi et al., 2022), we hold them fixed for simplicity and do not explore broader retuning here. Our test therefore asks whether learning-rate scaling alone is sufficient to align the sample-indexed curves $J_B(N)$ and recover the predicted $1/B$ scaling of $K^*(B)$. A failure of alignment may reflect either genuine statistical saturation or a limitation of this restricted protocol. In the following sections, Section 3.3 reports these training-side tests, while Section 4 separately evaluates the generation-side throughput gains attainable on fixed hardware.

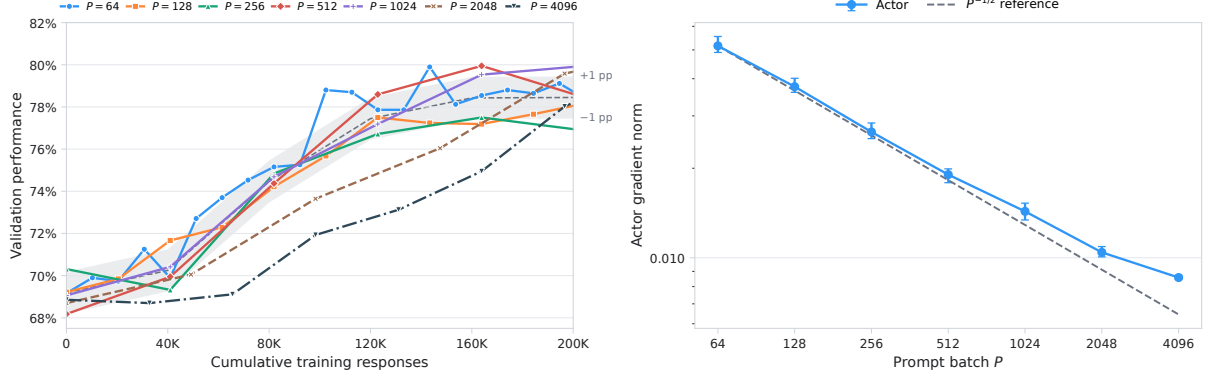


Figure 2: GRPO scaling at fixed group size $G = 8$. **Left:** the gray band is a ± 1 -percentage-point visualization band around the mean trajectory fitted at shared checkpoints spaced every 40K training responses using $P = 64$ –1024. These runs are approximately batch-size invariant over the measured window, whereas the held-out $P = 2048$ and $P = 4096$ trajectories deviate from the aligned family. **Right:** points and error bars show the actor-gradient-norm median and interquartile range over the common 40K–160K training-response window. The $P = 64$ –1024 medians follow the $P^{-1/2}$ reference closely, while the decline flattens at the largest batches, most clearly from $P = 2048$ to $P = 4096$.

3.3 Empirical Evidence

Related batch-scaling regularities appear in the empirical large-batch model of McCandlish et al. (2018) and in batch-size-invariant policy optimization by Hilton et al. (2022). We test whether approximate invariance also appears in LLM RL. All batch-size comparisons within each workload use a fixed hardware allocation: the machine configuration and accelerator count are held constant as the batch varies. Relative to the minibatch-based PPO/PPG setting of Hilton et al. (2022), we study single-pass policy-gradient methods: in both GRPO and PPO, each rollout batch supports one global optimizer step, without minibatch subdivision or rollout reuse.² With G responses per prompt, $N_{\text{train}} = kPG$ after k updates; this retained-response counter indexes all learning comparisons. PPO, the special case $G = 1$, provides complementary evidence. In our training infrastructure, both workloads use asynchronous partial rollouts with streaming generation, allowing generation concurrency to exceed the per-update training batch.³

Measurement protocol. We compare learning across configurations at the observed validation checkpoints. The ± 1 -percentage-point band serves only as a descriptive visual reference for assessing approximate alignment at these checkpoints. As an indirect proxy for gradient variance, we also report the logged gradient norm. By Jensen’s inequality, stochasticity can inflate its expected value; when gradient noise dominates, we therefore expect the norm to decrease as batch size grows.

GRPO setup. We train Qwen3-30B-A3B-Instruct-2507 across all GRPO configurations. The training data are a filtered and evaluated subset derived from DAPO-MATH-17K (Yu et al., 2025); the resulting dataset used by all runs contains 4,853 rows after filtering out solve-all and solve-none prompts under a 16K-token rollout budget. We report mean@32 performance averaged over AIME 2024 and AIME 2025. In the main sweep, $G = 8$ is fixed, so scaling the prompt batch P scales the retained training batch PG by the same factor. We sweep $P \in \{64, 128, 256, 512, 1024, 2048, 4096\}$ and initialize the learning rate from $(P_0, \eta_0) = (128, 10^{-6})$ using $\eta = \eta_0 \sqrt{P/P_0}$. This gives $\eta = \{1/\sqrt{2}, 1, \sqrt{2}, 2, 2\sqrt{2}, 4, 4\sqrt{2}\} \times 10^{-6}$. Our priority sampler weights prompts by their estimated pass rates, favoring values near 0.5. This increases the chance that a sampled group contains both successful and unsuccessful responses, thereby reducing rejection under outcome-based filtering—called dynamic sampling in DAPO (Yu et al., 2025). Accordingly, our empirical sample unit is a *training response*: $N = N_{\text{train}}$ counts responses retained for optimizer updates after filtering.

Fixed group size: invariance and its boundary. Figure 2 (left) shows the main GRPO sweep at fixed $G = 8$. After square-root learning-rate retuning, $P = 64$ –1024 follow approximately batch-size-invariant learning curves over the measured window. The $P = 2048$ and $P = 4096$ curves instead deviate from this aligned family, indicating that the invariant range has been exceeded.

²Under synchronized execution with zero rollout–training mismatch, this regime is on-policy.

³In our experiments, $B_{\text{train}} = PG$ and $B_{\text{gen}} = 2B_{\text{train}}$.

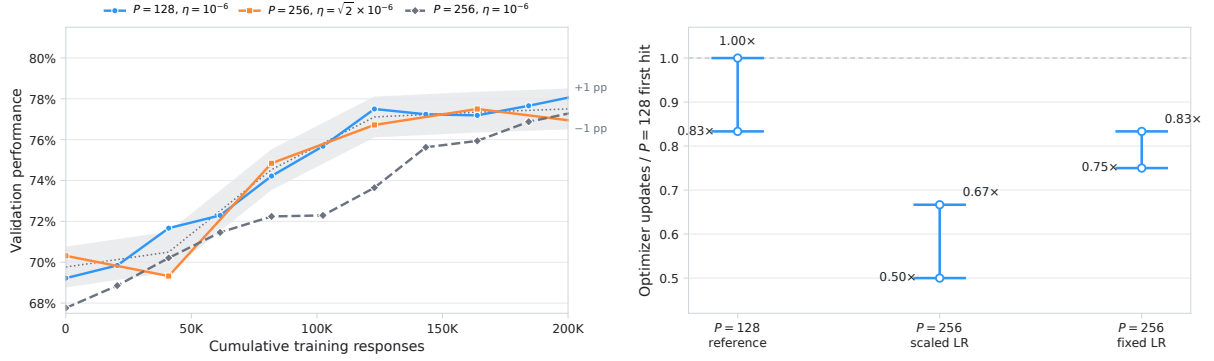


Figure 3: Learning-rate retuning improves sample-indexed alignment in GRPO. **Left:** square-root scaling keeps the $P = 256$ trajectory close to the $P = 128$ reference, whereas using the fixed $P = 128$ learning rate slows progress. **Right:** normalized optimizer updates required to reach the target; the capped intervals show checkpoint resolution. Exact invariance predicts $0.50\times$ when the batch doubles, which is consistent with the scaled-LR run but not the fixed-LR control.

Figure 2 (right) provides a consistent optimization-side explanation. Over $P = 64$ – 1024 , the actor-gradient-norm proxy has a fitted log-log slope of -0.468 , close to the $P^{-1/2}$ scaling expected when stochastic noise dominates. At the upper end, this scaling begins to flatten: doubling the batch from $P = 2048$ to $P = 4096$ reduces the median gradient norm to $0.823\times$, rather than the $0.707\times$ predicted by $P^{-1/2}$. The largest batches therefore no longer exhibit the same noise-dominated scaling, consistent with the loss of learning-curve invariance in the left panel.

Fixed-LR ablation: a larger batch is not automatically optimal. Figure 3 emulates a *common but inappropriate* practice: increasing the batch while keeping the learning rate fixed. In the left panel, the scaled-LR $P = 256$ run tracks the $P = 128$ reference at equal cumulative training responses, whereas the fixed-LR run learns more slowly and breaks batch-size invariance. The right panel tests Equation (3): exact invariance predicts that doubling P halves the optimizer updates to target. The scaled-LR crossing interval of 0.50 – $0.67\times$ contains the ideal $0.50\times$, whereas the fixed-LR interval of 0.75 – $0.83\times$ shows a smaller gain. Thus batch scaling alone yields weaker update-side scaling and is slower than the reference in the end-to-end accounting of Section 4.1.

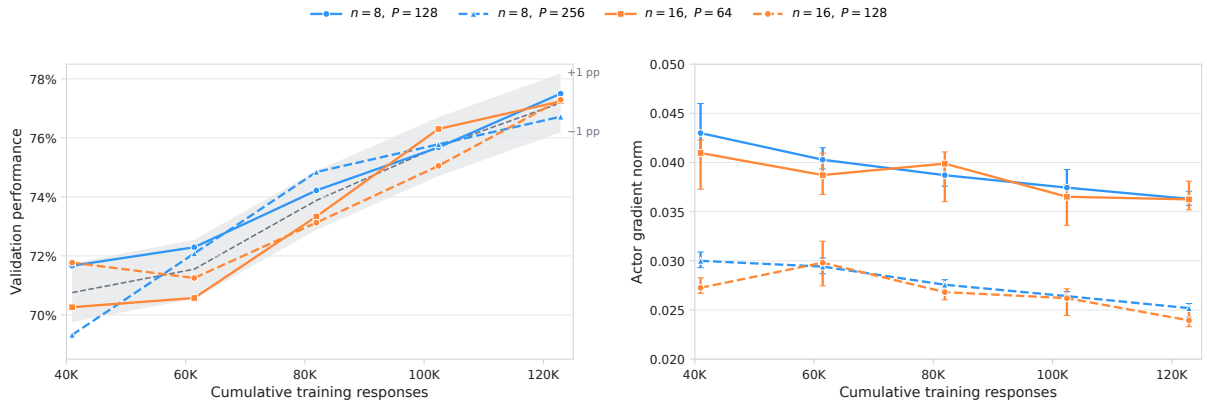


Figure 4: GRPO prompt-batch and group-size comparisons at matched training-response budgets over the common 40.96K–122.88K response window. The pairs $(G = 8, P = 128)$ versus $(G = 16, P = 64)$ and $(G = 8, P = 256)$ versus $(G = 16, P = 128)$ hold PG fixed at 1024 and 2048 retained responses per update, respectively. **Left:** validation trajectories approximately align when indexed by cumulative training responses. **Right:** actor-gradient-norm summaries are also similar within each matched pair.

What constitutes the batch size in GRPO? The preceding sweep changes the prompt batch P while keeping the group size G fixed. A GRPO update, however, contains P prompt groups with G responses per group, so its batch size could refer either to the number of prompts P or to the total response batch PG . To distinguish these possibilities, we vary G while holding PG fixed. Figure 4 shows that configurations with different (P, G) pairs but the same PG follow similar learning curves when indexed by cumulative

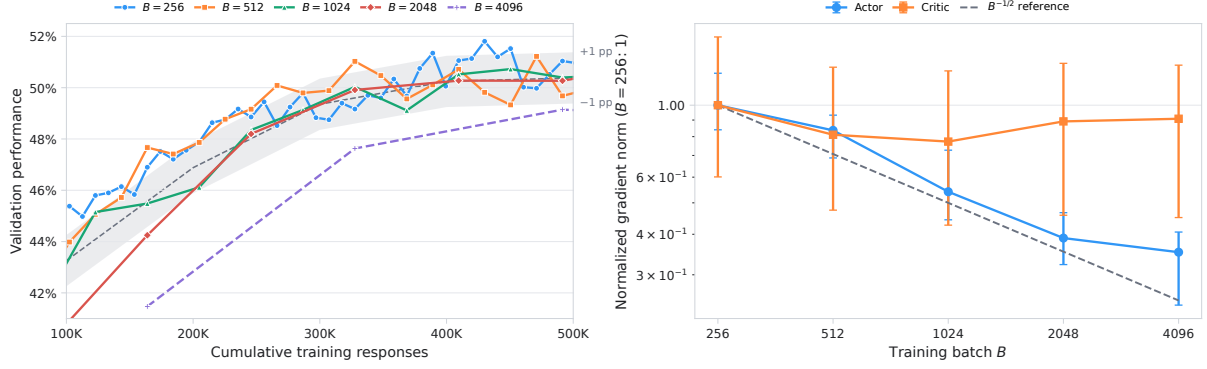


Figure 5: Training-side batch scaling for PPO. The cumulative-training-response axis excludes the $30 \times B$ responses collected during each run’s 30-step critic-only warm-up and is re-zeroed when actor optimization begins. **Left:** $B = 256$ – 2048 approximately align, whereas the held-out $B = 4096$ trajectory deviates. **Right:** normalized post-warm-up actor gradient norm decreases with batch, whereas the critic is comparatively batch-flat and noisier.

training responses; their gradient-norm summaries are also similar. Approximate batch-size invariance therefore continues to hold when the group size changes over the measured range. For this recipe, the relevant empirical batch and sample unit is the total number of training responses $P \times G$, rather than the prompt count P alone. We test $G \in \{8, 16\}$ and leave broader group sizes to future work.

PPO as complementary evidence. We next test bounded invariance in PPO, where each prompt contributes one response. We use an internal Hunyuan mixture-of-experts model and value model, each with 3B active parameters. The filtered set contains 13,405 mathematics, science, and logic examples. Starting from $(B_0, \eta_0) = (512, 10^{-6})$, we sweep $B \in \{256, 512, 1024, 2048, 4096\}$ with square-root learning-rate scaling; the critic learning rate is 10η . Evaluation reports held-out mean@4 on these domains. We exclude the $30 \times B$ responses generated during the 30-step critic-only warm-up and re-zero the actor-training sample axis for the optimization-side comparison.

PPO results and the boundary of alignment. Figure 5 shows approximate alignment for $B = 256$ – 2048 and deviation at $B = 4096$ (left), together with the corresponding optimization-side signal (right). Over $B = 256$ – 2048 , the actor-gradient-norm proxy has a fitted log-log slope of -0.471 , close to the $B^{-1/2}$ scaling expected in a noise-dominated regime. Doubling the batch from $B = 2048$ to $B = 4096$, however, reduces the median to only $0.905\times$, rather than the $0.707\times$ predicted by $B^{-1/2}$. This flattening is consistent with the loss of learning-curve invariance in the left panel. PPO is nevertheless more complex than GRPO because it jointly optimizes an actor and a value model, whose batch scales need not coincide.

Actor-critic batch scales. Definition 2 applies to policy learning: $K^*(B)$ counts actor updates required to reach the validation target. PPO additionally trains a value model with its own regression objective, so the actor and critic need not share the same batch scale. Surprisingly, Figure 5 (right) shows a marked asymmetry: as the batch grows, the actor gradient norm decreases, whereas the critic gradient norm remains nearly flat and substantially noisier. This suggests that larger batches benefit the actor and critic differently. Determining the critic’s own critical batch requires a separate analysis of value learning, which we leave to future work.

Summary and takeaway

- With Adam, square-root learning-rate scaling yields approximate invariance over bounded batch-size ranges in GRPO and PPO.
- A fixed learning rate breaks this alignment and should be retuned when the batch changes.
- For measured GRPO group sizes, total responses $P \times G$ are the batch and sample unit.
- PPO may have distinct critical training batch sizes for its actor and critic.

4 Generation–Training Asymmetry and Fixed-Hardware Efficiency

Section 3 identified a bounded regime in which larger batches preserve learning per response after learning-rate retuning. We now turn to the systems question: **on fixed hardware, can a larger batch convert this algorithmic invariance into a lower time-to-target?** Classical large-batch scaling relies on additional accelerators to process larger batches in parallel. Our setting instead keeps hardware fixed, so any wall-clock gain must come from higher realized throughput.

RL creates such an opportunity because every training response must first be generated autoregressively. The key systems mechanism is a *generation–training asymmetry*: doubling the rollout batch doubles the number of requested responses, but often increases generation time by substantially less than $2\times$. Autoregressive decoding is commonly underfilled at low concurrency, so additional active sequences share the cost of streaming model weights and increase response throughput. Training, by contrast, processes every token, so its arithmetic grows with training volume.

This asymmetry creates an acceleration opportunity even when the learning curve per response is unchanged. Within a batch-size-invariant range, higher response throughput directly reduces time-to-target; outside that range, the systems gain must be weighed against the samples-to-target penalty. Section 4.1 first presents fixed-hardware generation-stage evidence. Section 4.2 then explains why generation time can grow sublinearly with batch and where the gain saturates.

4.1 Empirical Evidence

The systems measurements below use the PPO and GRPO workloads introduced in Section 3.3 and a subset of the batch configurations evaluated there. Section 3.3 compared learning at equal cumulative responses; here we hold each workload’s hardware allocation fixed and measure how increasing the batch changes rollout-collection time, actor-update time, and realized response throughput.

PPO: response-collection throughput improves by $2.29\times$. Figure 6(a) reports generation-stage measurements from asynchronous partial-rollout implementations. In the Hunyuan A3B PPO runs, increasing B_{train} from 256 to 1024 multiplies responses per optimization boundary by $4\times$, while boundary-collection time grows only from 39 to 68 seconds ($68/39 = 1.74\times$). The resulting response-collection throughput improves by $4/(68/39) = 2.29\times$. This suggests that rollout collection is underfilled at the smaller PPO batch, so increasing the response batch can improve generation throughput even on fixed hardware.

GRPO: response-collection throughput improves by up to $1.36\times$. Figure 6(b) shows the same sublinear collection-time scaling in the Qwen3-30B-A3B GRPO runs. Over the common 40,960–286,720 training-response window at fixed $G = 8$, increasing P from 128 to 256 doubles the responses per optimization boundary but increases boundary-collection time by only $1.53\times$, yielding a $1.31\times$ response-throughput gain. Increasing P to 512 raises responses by $4\times$ while time grows by $2.93\times$, yielding a $1.36\times$ gain. The small additional improvement from $P = 256$ to $P = 512$ suggests diminishing generation-throughput returns over this range.

Actor-update time is approximately batch-proportional. The same GRPO measurements show a different scaling pattern for optimization. At fixed $G = 8$, doubling P from 128 to 256 increases mean actor-update time from 101.3 to 208.6 seconds ($2.06\times$) under the scaled learning rate; the fixed-LR control gives a similar $1.98\times$ increase. Thus, over the measured range, actor-update time is close to linear in the training batch, in contrast to the sublinear growth of rollout-collection time.

4.2 Sublinear Generation Cost and the Critical Generation Batch

The empirical asymmetry in Section 4.1 comes from how fixed hardware executes the two stages. Autoregressive generation advances each sequence by one token at a time; token $u + 1$ cannot be computed before token u , so a single sequence exposes little parallel work. At low active concurrency, each layer applies large weight matrices to only a few token vectors. Decode therefore has low arithmetic intensity and is often memory-bandwidth-bound. Increasing the number of concurrent sequences widens these batched operations and amortizes each weight transfer over more generated tokens, raising hardware utilization and throughput (Williams et al., 2009; Pope et al., 2023; Zhong et al., 2024).

Training exposes substantially more parallelism. Once the responses are available, the forward and backward passes process many token positions across many sequences simultaneously. The resulting large matrix–matrix operations have higher arithmetic intensity and typically keep the arithmetic units much better utilized. On fixed hardware, increasing the training batch therefore adds approximately

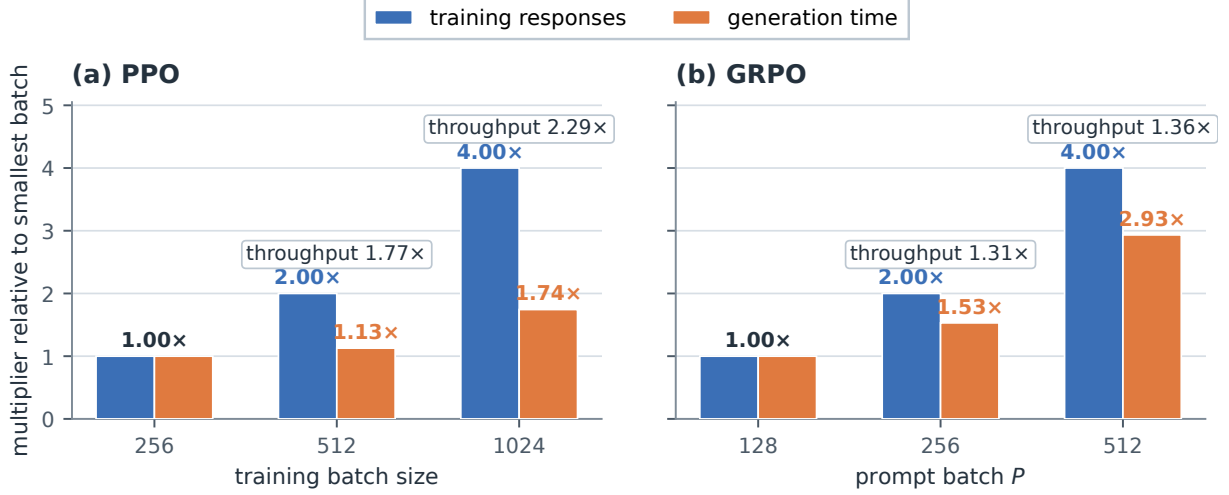


Figure 6: Fixed-hardware generation scaling for PPO and GRPO. Blue bars show training responses collected per optimization boundary and orange bars show boundary-collection time, each normalized to the smallest batch. Boxed labels report the corresponding generation-stage response throughput.

proportional FLOPs rather than unlocking the same unused parallelism. Consistent with this distinction, Section 4.1 observes sublinear rollout-collection time but nearly batch-proportional actor-update time.

We use B_{gen} for rollout responses available to generation and B_{train} for retained responses consumed by an optimizer update. We therefore model training with a batch-proportional term and generation with an amortized term:

$$t_{\text{gen}}(B_{\text{gen}}) \approx d_0 + d_1 B_{\text{gen}}, \quad t_{\text{train}}(B_{\text{train}}) \approx \gamma B_{\text{train}}. \quad (4)$$

Here d_0 is the batch-independent generation latency, d_1 is the batch-proportional generation-time coefficient, and γ is the batch-proportional training-time coefficient; d_1 and γ have units of time per response under the chosen batch unit. The intercept captures work amortized across concurrent responses. Generation time can therefore grow sublinearly, whereas training time scales roughly with batch.

These amortization gains eventually saturate as batch-dependent arithmetic, KV-cache traffic, response-length imbalance, and memory capacity become limiting (Yu et al., 2022; Pope et al., 2023; Kwon et al., 2023). We call the smallest logical rollout batch that captures most of the attainable generation throughput the *critical generation batch*; Appendix A derives this saturation with a decode-step Roofline model and relates the logical rollout batch to the active batch seen by decode kernels.

Definition 3 (Critical generation batch). *For a fixed model, decoding engine, length distribution, and hardware allocation, let $q_{\text{gen}}(B)$ be aggregate generation throughput at batch B , and let $q_{\text{gen}}^{\text{max}}$ be its plateau. For a small tolerance $\delta > 0$, the critical generation batch $B_{\text{gen}}^{\text{crit}}$ is the smallest B satisfying*

$$q_{\text{gen}}(B) \geq (1 - \delta) q_{\text{gen}}^{\text{max}}.$$

Practical implication. Definition 3 marks the systems knee beyond which most attainable generation throughput has already been realized; it is not by itself the wall-clock-optimal batch. Unlike $B_{\text{train}}^{\text{crit}}$, $B_{\text{gen}}^{\text{crit}}$ depends on the model, decoding engine, response-length distribution, and hardware allocation, and must be profiled for the target deployment. In practice, sweep B_{gen} or decode concurrency, verify stable throughput and KV-cache feasibility, and stop once additional batching yields little throughput gain. The selected configuration lowers time-to-target only if its end-to-end throughput gain exceeds the samples-to-target penalty. Section 5 combines these generation- and training-side constraints.

Summary and takeaway

- On fixed hardware, increasing the response batch improves generation throughput by up to $2.29\times$ in PPO and $1.36\times$ in GRPO.
- Generation and training scale asymmetrically: larger active batches amortize model-weight traffic during decoding, whereas training work grows approximately with processed tokens.
- Generation time therefore often grows sublinearly with the response batch, until another systems bottleneck causes throughput to plateau.

5 Operating Regimes and Practical Guidance

The preceding sections separately measured learning per sample and systems throughput. We now combine them through $T = N^*/q$: a larger batch reduces time-to-target only when its throughput gain exceeds its samples-to-target penalty r_N . In principle, this tradeoff can take one of two forms, depending on whether the generation or training critical batch is reached first.

Two orderings of the critical batches. Which ordering arises depends on the hardware, model size, and dataset size. When one coupled batch B scales both generation and training, the two orderings produce two practical regimes. Figure 7 illustrates the resulting $T(B) = N^*(B)/q(B)$; neither critical batch alone determines the minimizing batch B^* .

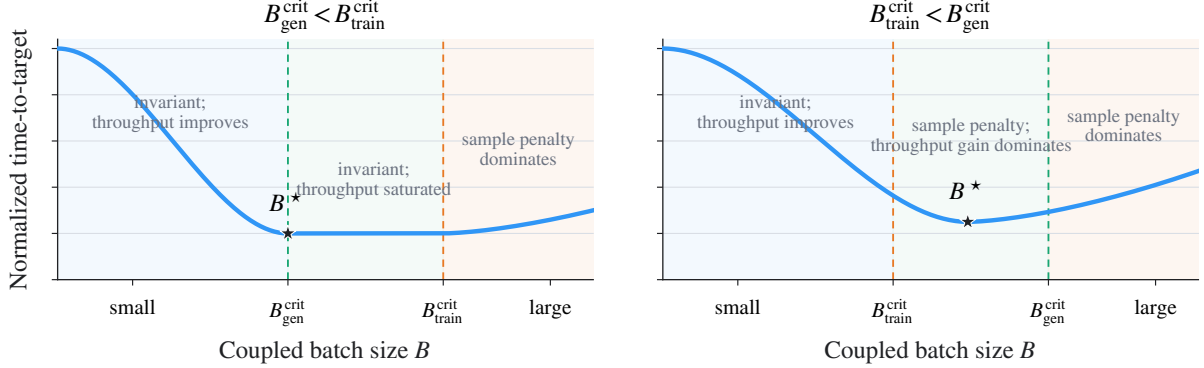


Figure 7: Schematic normalized time-to-target for a synchronously coupled batch B . The panels show the two possible orderings of $B_{\text{train}}^{\text{crit}}$ and $B_{\text{gen}}^{\text{crit}}$; B^* marks the wall-clock optimum. The curves are schematic rather than fitted to our experiments.

- **Left: generation knee first.** Throughput plateaus while $r_N \approx 1$, creating a broad near-optimal region. Further scaling provides little systems benefit and eventually raises time-to-target as the sample penalty becomes material.
- **Right: training knee first.** The sample penalty begins while generation throughput is still improving. When the throughput gain is larger, B^* can lie beyond $B_{\text{train}}^{\text{crit}}$ but before $B_{\text{gen}}^{\text{crit}}$.

Measured GRPO operating point. Table 1 applies this accounting to the fixed- $G = 8$ GRPO sweep. It reports both sides of the tradeoff for every configuration: r_N measures the samples-to-target penalty, while q/q_{128} measures the realized throughput gain relative to $P = 128$.

Table 1: GRPO time-to-target accounting at fixed $G = 8$ and $J^* = 77\%$. K^* is the evaluated optimizer update used for accounting, and $J(K^*)$ is its validation value. N_{train}^* is reported in thousands of retained training responses, $q = N_{\text{train}}^*/T$, and all ratios use $P = 128$ as the reference. The final row is the fixed-LR $P = 256$ control; all other rows use square-root LR scaling.

P	$\eta/10^{-6}$	$J(K^*)$	K^*	N_{train}^* (K)	T (h)	r_N	q/q_{128}
64	$1/\sqrt{2}$	78.80	200	102.40	13.56	0.83	0.73
128	1	77.50	120	122.88	11.90	1.00	1.00
256	$\sqrt{2}$	76.72	60	122.88	9.54	1.00	1.25
512	2	78.59	30	122.88	8.77	1.00	1.36
1024	$2\sqrt{2}$	77.19	15	122.88	8.42	1.00	1.41
2048	4	79.58	12	196.61	14.68	1.60	1.30
4096	$4\sqrt{2}$	78.02	6	196.61	14.00	1.60	1.36
256	1 fixed	77.40	100	204.80	16.93	1.67	1.17

Figure 8 visualizes the same accounting. The measured sweep follows the generation-knee-first pattern in Figure 7: normalized time-to-target falls to 0.71 at $P = 1024$ while r_N remains near one, then rises to 1.23 and 1.18 at $P = 2048$ and $P = 4096$ once r_N reaches 1.60. The fixed-LR $P = 256$ control takes $1.42\times$ as long as the reference because its $1.17\times$ throughput gain does not offset its $1.67\times$ sample penalty.

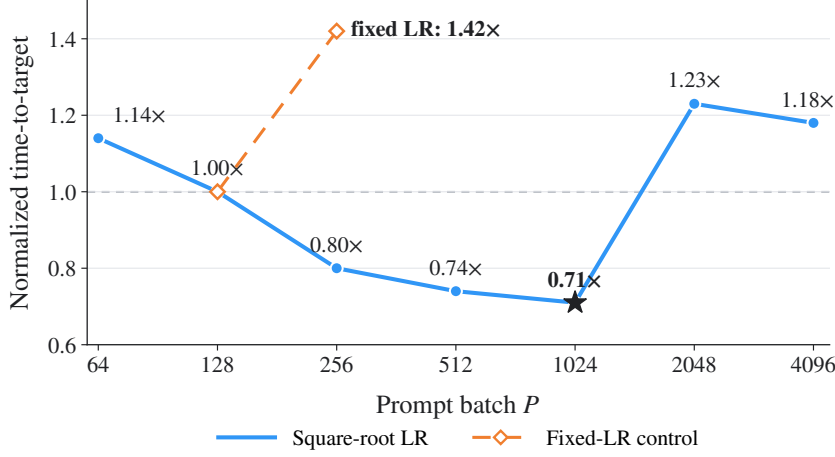


Figure 8: Measured GRPO time-to-target sweep at fixed $G = 8$ and $J^* = 77\%$, using the accounting in Table 1. Blue shows the square-root-LR configurations, orange shows the fixed-LR control, and the star marks the minimum measured time-to-target. All times are normalized to the $P = 128$ reference.

Principle 1 (Align, then accelerate). *First retune batch-dependent hyperparameters to preserve learning per sample. Then increase batching only while its throughput gain exceeds its samples-to-target penalty.*

Two-stage tuning procedure. Algorithm 1 provides a practical procedure for applying Principle 1. It first establishes training-side alignment and then searches for systems-side throughput gains.

Algorithm 1 Two-stage batch tuning

Input: Reference (B_0, η_0) , candidate training batches $\mathcal{B}$, and target J^*

- 1: **for** $B_{\text{train}} \in \mathcal{B}$ **do**
 - 2: For Adam-type optimizers, initialize $\eta \leftarrow \eta_0 \sqrt{B_{\text{train}}/B_0}$; retune as needed.
 - 3: Run a sample-matched learning curve and estimate r_N at J^* .
 - 4: Increase B_{gen} or decode concurrency until throughput plateaus or a feasibility limit is reached.
 - 5: Measure stable end-to-end throughput q using the same sample counter.
 - 6: **return** the feasible configuration maximizing q/r_N .
-

For each candidate training batch, the first stage initializes and retunes batch-dependent hyperparameters, then estimates r_N from a sample-matched learning curve. We use the square-root learning-rate initialization for Adam-type optimizers; other optimizers require their corresponding batch-scaling rule. The second stage increases generation concurrency and measures stable end-to-end throughput q . The procedure returns the feasible configuration maximizing q/r_N .

6 Related Work

The closest literature addresses three quantities that are often conflated: the training batch used to estimate an update, the number of rollouts used to explore each prompt, and the active concurrency used to execute generation. We keep these quantities separate and reconnect them only through time-to-target.

6.1 Training-Batch Scaling and Invariance

Classical critical-batch work is motivated by data-parallel training when examples are already available: increasing the batch can reduce the number of sequential optimizer updates, but this benefit exhibits diminishing returns. McCandlish et al. (2018) relate the transition to the gradient noise scale, while Shallue et al. (2019) show empirically that useful parallelism is workload-dependent and that batch comparisons require fair hyperparameter tuning. The critical batch can also change over the course of language-model training (Merrill et al., 2025).

Hilton et al. (2022) define policy optimization as batch-size invariant when hyperparameter retuning preserves behavior as a function of processed samples. They show that standard PPO is not automatically invariant because its old policy serves both behavior-correction and proximal roles. Their PPO-EWMA

and PPG-EWMA constructions decouple these roles, use square-root learning-rate scaling for Adam, and require a single policy epoch for their invariance construction; they separately show that the decoupled objective better tolerates artificially stale data. The SDE analysis of [Malladi et al. \(2022\)](#) provides a complementary account of the Adam scaling rule.

Relative to this work, our contribution is an empirical transfer test in a different combined regime rather than a new invariance construction. We study modern LLM RL using single-pass policy-gradient methods: both GRPO and PPO perform one global update per rollout batch, without minibatch steps or rollout reuse, and both workloads run with asynchronous partial rollouts and streaming generation. Without introducing the EWMA proximal-policy mechanism of [Hilton et al. \(2022\)](#), we test how far Adam learning-rate retuning alone aligns their sample-indexed learning curves. The resulting evidence supports bounded approximate invariance in this LLM RL setting and connects it to measured systems throughput and time-to-target.

6.2 Rollout Scaling and Generation Efficiency

Our framework is a fixed-hardware extension of classical critical-batch analysis ([McCandlish et al., 2018](#); [Shallue et al., 2019](#)): it compares the statistical cost of a larger batch, measured by samples-to-target, with its systems benefit, measured by end-to-end throughput. [Piché et al. \(2025\)](#) use a similar effectiveness-throughput decomposition to study pipeline scheduling and data staleness, whereas we use it to characterize batch-size selection under fixed hardware.

The work by [Hu et al. \(2025\)](#) is the closest to our outcome-filtering setting. BroRL increases the number of rollouts per prompt from 16 to 512, making a group less likely to be rejected as uniformly correct or uniformly incorrect. It reports both higher dynamic-sampling acceptance and higher generation throughput, and interprets the algorithmic benefit more broadly as improved exploration. The key conceptual difference is that we formally study batch-size invariance. In our framework, both the prompt batch P and group size G are batch dimensions: after appropriate retuning, changing either can approximately preserve sample-indexed learning. BroRL instead emphasizes a mechanism specific to dynamic sampling, where a larger rollout count per prompt reduces all-correct and all-incorrect groups and thereby increases the fraction of generated responses retained for training. Related methods, including Knapsack RL, AR3PO, and VIP, likewise improve sampling efficiency by allocating prompt-dependent rollout budgets rather than increasing G uniformly ([Li et al., 2026](#); [Zhang et al., 2025](#); [Nguyen et al., 2026](#)). These methods improve the learning value obtained from a fixed rollout budget rather than establishing invariance of learning per retained response across batch configurations.

The IsoCompute Playbook optimizes how a fixed generated-rollout budget is allocated across problems per batch, rollouts per problem, and sequential updates ([Cheng et al., 2026](#)). Its preference for larger rollout groups partly reflects fewer zero-signal groups on hard problems, but also solution sharpening and reduced cross-problem interference; unlike our retained-response analysis, it does not separate sampling yield from learning per retained response.

On the systems side, the Roofline model explains why increasing active decode concurrency can amortize weight traffic ([Williams et al., 2009](#)), while KV-cache management constrains feasible concurrency ([Kwon et al., 2023](#)). Frameworks that overlap or reschedule rollout and training stages change the attainable end-to-end throughput ([Sheng et al., 2025](#); [Fu et al., 2025](#); [Piché et al., 2025](#)); they do not change the criterion that the throughput gain must exceed the sample penalty.

7 Conclusion

This paper studies when larger batches help scale LLM reinforcement learning by separating learning per sample from systems throughput. After batch-dependent hyperparameters are retuned, batch-size invariance makes different physical batches comparable through approximately aligned sample-indexed learning curves over a bounded range. Our GRPO and PPO experiments support this property, while the fixed-learning-rate control and the largest-batch runs show that it is not automatic or unbounded. Generation-training asymmetry provides the complementary systems opportunity: increasing rollout concurrency can improve response throughput without proportionally increasing generation time, and our tuned GRPO configurations translate this gain into lower time-to-target. The practical takeaway is therefore simple: retune the learning rate and other batch-dependent hyperparameters whenever the training batch changes; once the recipe is stable, increase the batch only while its end-to-end throughput gain exceeds its samples-to-target penalty.

Because our experiments retune only the learning rate, future work should derive scaling rules for other batch-dependent hyperparameters and pursue tighter batch-size invariance as a basis for algorithm design, comparison, and transfer. It should also measure how the training and generation critical batches

shift across models, algorithms, hardware, and execution modes, and clarify the coupled actor-critic batch scales in PPO.

Author Contributions

Ziniu Li led the project and developed the central connection between batch-size invariance and generation-training computational asymmetry. Jinbo Wang, Guanhua Huang, and Alex Chen contributed to the conceptual discussions. In particular, Jinbo Wang noted the expected batch scaling of the diagnostic actor gradient norm and the differing behavior of actor and value-model gradients in PPO. Feiyuan Zhang and Pengbo Li analyzed and interpreted the computational asymmetry between rollout generation and training. Alex Chen supervised the project.

References

- Zhoujun Cheng, Yutao Xie, Yuxiao Qu, Amrith Setlur, Shibo Hao, Varad Pimpalkhute, Tongtong Liang, Feng Yao, Zhengzhong Liu, Eric Xing, Virginia Smith, Ruslan Salakhutdinov, Zhiting Hu, Taylor Killian, and Aviral Kumar. IsoCompute Playbook: Optimally scaling sampling compute for LLM RL. *arXiv preprint arXiv:2603.12151*, 2026.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Jacob Hilton, Karl Cobbe, and John Schulman. Batch size-invariance for policy optimization. In *Advances in Neural Information Processing Systems*, volume 35, pp. 17086–17098, 2022.
- Jian Hu, Mingjie Liu, Ximing Lu, Fang Wu, Zaid Harchaoui, Shizhe Diao, Yejin Choi, Pavlo Molchanov, Jun Yang, Jan Kautz, and Yi Dong. BroRL: Scaling reinforcement learning via broadened exploration. *arXiv preprint arXiv:2510.01180*, 2025.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 29128–29163. PMLR, 2024.
- Ziniu Li, Congliang Chen, Tianyun Yang, Tian Ding, Ruoyu Sun, Ge Zhang, Wenhao Huang, and Zhi-Quan Luo. Knapsack RL: Compute-efficient reinforcement learning via heterogeneous rollout allocation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026.
- Sadhika Malladi, Kaifeng Lyu, Abhishek Panigrahi, and Sanjeev Arora. On the sdes and scaling rules for adaptive gradient algorithms. In *Advances in Neural Information Processing Systems*, volume 35, pp. 7697–7711, 2022.
- Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.
- Will Merrill, Shane Arora, Dirk Groeneveld, and Hanna Hajishirzi. Critical batch size revisited: A simple empirical approach to large-batch language model training. In *Advances in Neural Information Processing Systems*, volume 38, pp. 116936–116959, 2025.
- Hieu Trung Nguyen, Bao Nguyen, Wena Ma, Yuzhi Zhao, Ruifeng She, and Viet Anh Nguyen. Adaptive rollout allocation for online reinforcement learning with verifiable rewards. In *International Conference on Learning Representations*, 2026.

-
- OpenAI. OpenAI o1 System Card. <https://openai.com/index/openai-o1-system-card/>, 2024. Updated December 5, 2024.
- Alexandre Piché, Ehsan Kamalloo, Rafael Pardini, Xiaoyin Chen, and Dzmitry Bahdanau. PipelineRL: Faster on-policy reinforcement learning for long sequence generation. *arXiv preprint arXiv:2509.19128*, 2025.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the effects of data parallelism on neural network training. *Journal of Machine Learning Research*, 20(112):1–49, 2019.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. Don’t decay the learning rate, increase the batch size. In *International Conference on Learning Representations*, 2018.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Gal Kaplun, Sham M. Kakade, and Boaz Barak. Beyond implicit bias: The insignificance of SGD noise in online learning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 49698–49716, 2024.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pp. 521–538, 2022.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yuheng Zhang, Wenlin Yao, Changlong Yu, Yao Liu, Qingyu Yin, Bing Yin, Hyokun Yun, and Lihong Li. Improving sampling efficiency in RLVR through adaptive rollout and response reuse. *arXiv preprint arXiv:2509.25808*, 2025.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pp. 193–210, 2024.

A Analytical Details for Generation–Training Asymmetry

This appendix collects the analytical details omitted from Section 4.2. For a synchronous comparison with a fixed response-length distribution, set $B_{\text{gen}} = B_{\text{train}} = B$ and consider a fixed budget of N responses. Under Equation (4), total generation and training time are approximately

$$T_{\text{gen}}(N; B) \approx \frac{N}{B} t_{\text{gen}}(B) \approx N \left(\frac{d_0}{B} + d_1 \right),$$

$$T_{\text{train}}(N; B) \approx \frac{N}{B} t_{\text{train}}(B) \approx N\gamma.$$

The repeated weight-streaming term decreases as d_0/B , while the dominant training arithmetic remains approximately constant over the fixed response budget.

The aggregate generation batch in the main text is not necessarily the batch dimension seen by an individual decode kernel. At decode step u , let $B_{\text{act}}(u) \leq B_{\text{gen}}$ be the number of unfinished sequences that contribute a new token. The distribution of $B_{\text{act}}(u)$ over a rollout boundary is determined by admission, scheduling, and the response-length distribution. Increasing B_{gen} improves generation throughput only insofar as it raises this active concurrency.

A decode-step Roofline calculation makes the generation-side amortization explicit. At step u , the system produces one new token for each of the $B_{\text{act}}(u)$ active sequences. The same weights are applied to these tokens in a batched operation, so weight traffic is largely independent of $B_{\text{act}}(u)$, whereas arithmetic and KV-cache traffic grow with it (Pope et al., 2023; Zhong et al., 2024). Let M_W be the weight bytes transferred at a decode step, m_u the additional per-sequence memory traffic at step u , including KV-cache reads and writes, f_{tok} the FLOPs per generated token, W_{HBM} the sustained memory bandwidth, and F_{eff} the sustained arithmetic throughput. The Roofline model gives (Williams et al., 2009)

$$t_{\text{dec}}(u) \gtrsim \max \left\{ \frac{M_W + B_{\text{act}}(u)m_u}{W_{\text{HBM}}}, \frac{B_{\text{act}}(u)f_{\text{tok}}}{F_{\text{eff}}} \right\}, \quad q_{\text{tok}}(u) = \frac{B_{\text{act}}(u)}{t_{\text{dec}}(u)}. \quad (5)$$

Here $t_{\text{dec}}(u)$ is decode-step latency and $q_{\text{tok}}(u)$ is the corresponding token throughput. At small $B_{\text{act}}(u)$, weight transfer dominates, so $t_{\text{dec}}(u) \approx M_W/W_{\text{HBM}}$ changes little as active concurrency increases, while $q_{\text{tok}}(u)$ grows nearly linearly. At larger active batches, the batch-dependent compute or KV-cache term eventually dominates. Step time then grows with active concurrency, and throughput approaches a plateau instead of continuing to scale linearly.

Equation (5) also gives a hardware lower bound for each decode step. The corresponding decode arithmetic intensity and ridge point are

$$\text{AI}_{\text{dec}}(u) = \frac{B_{\text{act}}(u)f_{\text{tok}}}{M_W + B_{\text{act}}(u)m_u}, \quad \rho = \frac{F_{\text{eff}}}{W_{\text{HBM}}}.$$

Decode is bandwidth-bound when $\text{AI}_{\text{dec}}(u) < \rho$. Increasing the active batch amortizes M_W and raises arithmetic intensity until the ridge point or another limit is reached.

Figure 9 instantiates the Roofline model for a 110B-parameter dense transformer with tensor-parallel degree $\text{TP} = 4$, using parameters representative of a high-bandwidth accelerator. Panel (a) uses a forward-pass GEMM proxy with 40K tokens per sequence, whereas panel (b) averages decode steps over an 8K-token prompt and a 32K-token response. Accordingly, the plotted symbol B denotes the local training batch in panel (a) and the active decode batch in panel (b). The common sweep from 1 to 32 provides a controlled comparison; it does not imply that $B_{\text{train}} = B_{\text{act}}$ in an asynchronous deployment. The resulting values are analytical per-layer lower bounds. They are not end-to-end measurements and do not estimate the critical batch.

Increasing the local training batch from 1 to 32 raises the compute-bound training-proxy latency by $32\times$, leaving ideal aggregate training throughput unchanged. By contrast, increasing the active decode batch by $32\times$ raises the I/O-bound decode latency by only $2.32\times$, because batch-independent model-weight traffic is amortized across the batch. This yields a $13.8\times$ ideal aggregate decode throughput gain. The figure illustrates this mechanism; the absolute times remain configuration-dependent.

Operationally, $B_{\text{gen}}^{\text{crit}}$ is the logical batch whose active-concurrency distribution brings aggregate throughput near its plateau, rather than a universal kernel batch. KV-cache capacity imposes a separate feasibility limit. For context length S , number of transformer layers L_{layer} , number of KV heads assigned to each device after tensor-parallel sharding H_{KV} , head dimension d_h , and bytes per cached scalar s_{KV} , the per-sequence KV-cache footprint and an optimistic memory-feasible active batch are

$$m_{\text{KV}}(S) = 2L_{\text{layer}}H_{\text{KV}}d_hSs_{\text{KV}}, \quad B_{\text{KV}} = \left\lfloor \frac{\alpha C_{\text{mem}} - M_{\text{weights}} - M_{\text{runtime}}}{m_{\text{KV}}(S)} \right\rfloor,$$

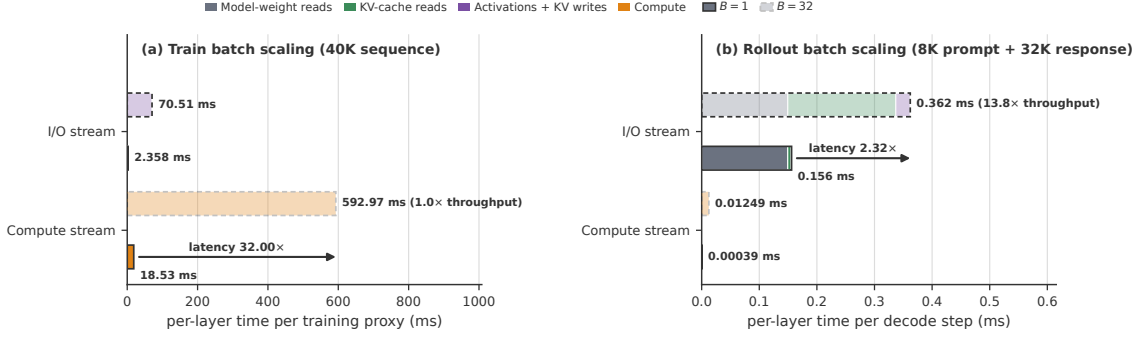


Figure 9: Analytical per-layer stream decomposition under batch scaling for (a) a training proxy and (b) rollout in a representative fixed-hardware setting. Solid and light dashed bars denote $B = 1$ and $B = 32$, respectively. Assuming ideal overlap between I/O and compute, latency is modeled as the maximum of the two; parenthesized values report ideal aggregate throughput relative to $B = 1$.

where C_{mem} is device memory capacity, M_{weights} is the resident model-weight footprint, $\alpha \in (0, 1)$ is the fraction available to the serving engine, and M_{runtime} reserves memory for activations, workspaces, fragmentation, and runtime state. Under tensor parallelism, all terms are per-device after sharding. Here M_{weights} is a resident-capacity term, whereas M_W in Equation (5) denotes weight traffic during a decode step. Paging reduces fragmentation but not the linear growth with context length (Kwon et al., 2023). A feasible logical generation batch must induce $B_{\text{act}}(u) \leq B_{\text{KV}}$ throughout decoding, subject to runtime admission and stability constraints; this memory limit may prevent aggregate generation throughput from reaching its plateau.